



TILERA[®]

***MULTICORE
DEVELOPMENT
ENVIRONMENT
TRIO
PROGRAMMER'S GUIDE***

DOCUMENT RELEASE 1.20
DOC. No. UG503
AUGUST 2014
TILERA CORPORATION

Copyright © 2011-2014 Tilara® Corporation. All rights reserved. Printed in the United States of America.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, except as may be expressly permitted by the applicable copyright statutes or in writing by the Publisher.

The following are registered trademarks of Tilara Corporation: Tilara and the Tilara logo.

The following are trademarks of Tilara Corporation: Embedding Multicore, The Multicore Company, Tile Processor, TILE Architecture, TILE64, TILEPro, TILEPro36, TILEPro64, TILEExpress, TILEExpress-64, TILEExpressPro-64, TILEExpress-20G, iMesh, TileDirect, TILExtreme-Gx, TILExtreme-Gx Duo, TILEmpower, TILEmpower-Gx, TILEmpower-Gx36, TILEmpower-Gx72, TILEncore, TILEncorePro, TILEncore-Gx, TILEncore-Gx9, TILEncore-Gx16, TILEncore-Gx36, TILEncore-Gx72, TILE-Gx, TILE-Gx9, TILE-Gx16, TILE-Gx36, TILE-Gx72, TILE-Gx8072, TILE-Gx3000, TILE-Gx5000, TILE-Gx8000, TILE-Gx8009, TILE-Gx8016, TILE-Gx8036, TILE-Gx3036, DDC (Dynamic Distributed Cache), Multicore Development Environment, Gentle Slope Programming, TMC (Tilara Multicore Components), hardwall, Zero Overhead Linux (ZOL), MiCA (Multicore iMesh Coprocessing Accelerator), and mPIPE (multicore Programmable Intelligent Packet Engine). All other trademarks and/or registered trademarks are the property of their respective owners.

Third-party software: The Tilara IDE makes use of the BeanShell scripting library. Source code for the BeanShell library can be found at the BeanShell website (<http://www.beanshell.org/developer.html>).

This document contains advance information on Tilara products that are in development, sampling or initial production phases. This information and specifications contained herein are subject to change without notice at the discretion of Tilara Corporation.

No license, express or implied by estoppels or otherwise, to any intellectual property is granted by this document. Tilara disclaims any express or implied warranty relating to the sale and/or use of Tilara products, including liability or warranties relating to fitness for a particular purpose, merchantability or infringement of any patent, copyright or other intellectual property right.

Products described in this document are NOT intended for use in medical, life support, or other hazardous uses where malfunction could result in death or bodily injury.

THE INFORMATION CONTAINED IN THIS DOCUMENT IS PROVIDED ON AN "AS IS" BASIS. Tilara assumes no liability for damages arising directly or indirectly from any use of the information contained in this document.

Publishing Information:

Document Number	UG503
Document Release Number	1.20
Date	01 August 2014

Contact Information:

Tilara Corporation
Information info@tilera.com
Web Site http://www.tilara.com

CONTENTS

PREFACE	V
----------------------	----------

CHAPTER 1 UNDERSTANDING TRIO

1.1 Overview of TRIO	1
1.2 gxpci API	2
1.3 gxio_trio API	2
1.4 TRIO Context Object and GXPCI Context Object	3
1.5 gxpci Data Structures and Typedefs	3
1.6 Linking with the gxpxi and gxio Libraries	3
1.7 Documentation for TRIO	3

CHAPTER 2 PROGRAMMING TRIO APPLICATIONS

2.1 TRIO Application Tasks and Routines	5
2.2 Transmitting Data using GXPCI Packet Queue API	5
2.3 Transmitting Data using the GXPCI Raw DMA API	6

CHAPTER 3 USING THE LOW-LEVEL GXIO_TRIO API

3.1 Overview of gxio_trio	7
3.2 TRIO Ingress Hardware Resources	7
3.2.1 TRIO Mapped Memory (MapMem)	7
3.2.2 TRIO Scatter Queues	8
3.3 TRIO Egress Hardware Resources	8
3.3.1 TRIO Programmed I/O (PIO)	8
3.3.2 TRIO Push and Pull DMA	8

3.4 gxio_trio Data Structures and Typedefs	8
3.5 Documentation for gxio_trio	8

CHAPTER 4 EXAMPLE APPLICATIONS

4.1 Chip-to-Chip Zero Copy Queue Example	11
4.2 Character Stream Example	12
4.3 Chip-to-Chip Transfer Queue Example	12
4.4 Host NIC Example	13
4.5 Host Packet Queue Example	13
4.6 Raw DMA Example	13
4.7 Memory Map Region Interrupts (MMI) Example	14
4.8 Zero Copy Example	15
4.9 Push DMA Example	15
4.10 Packet Queue with Linux Tap Example	16

INDEX	19
--------------------	-----------

PREFACE

About This Guide

This guide describes the Transaction I/O (TRIO) capability for the TILE-Gx™ incorporating industry-standard PCIe technology and StreamIO technology from Tiler® Corporation from a software perspective.

This guide applies to MDE Release 4.2 and later.

Intended Audience

This guide is intended for use by TILE-Gx™ processor application developers.

Note: This guide assumes that you are already familiar with the chapter describing the “PCIe Controller Architecture (TRIO)” in the *Tile Processor and I/O Device Guide for the TILE-Gx Family of Processors* (UG404).

Changes from the Previous Release

The major changes in this guide from MDE 4.1 are:

- [Chapter 4: Example Applications](#) includes the descriptions of the following example applications:
 - [Section 4.3 “Chip-to-Chip Transfer Queue Example”](#)
 - [Section 4.10 “Packet Queue with Linux Tap Example”](#)

Document Organization

This guide is organized as follows:

- [Chapter 1: Understanding TRIO](#)
Provides an introduction to TRIO.
- [Chapter 2: Programming TRIO Applications](#)
Describes the tasks an application needs to perform to use TRIO features.
- [Chapter 3: Using the Low-Level gxio_trio API](#)
Describes how to use TRIO features at a low level. Note: For advanced users only.
- [Chapter 4: Example Applications](#) describes the TRIO examples in the examples directory at `$TILER_ROOT/examples/trio/`.
- [Chapter 5: Frequently Asked Questions \(FAQs\)](#)
Provides FAQs about TRIO and how to use it.

This guide also includes an [Index](#).

MDE Documentation

Note: `TILERA_ROOT` is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is required by many Tiler tools and examples. We recommend that you use the `tile-env` script described in *Getting Started with the Multicore Development Environment* (UG504) to set it correctly.

Tilera MDE Document Index

The Tilera MDE Document Index provides links to online copies of the MDE documentation, in both HTML and PDF formats:

```
$TILERA_ROOT/doc/index.html
```

Location of PDF Files in the Kit

The PDF versions of MDE documents can be found at this location:

```
$TILERA_ROOT/doc/pdf/
```

Man Pages and Info Pages

To view a man page for an x86 tool, run:

```
$ tile-man toolname
```

To view a man page for a Tile tool, run this on a TILE system:

```
$ man toolname
```

To view an info page for a Tile tool, run this on a TILE system:

```
$ info toolname
```

IDE Online Help

The IDE has its own [Tilera IDE Online Documentation](#), which you can access when you use the IDE.

To get to the online help:

- Start `tile-eclipse`.

Select the menu command [Help → Help Contents](#).

- When the online help browser is displayed, select the document [Tilera IDE Online Documentation](#) in the left navigation pane, if it is not already selected.

Technical or Customer Support

You can reach Tilera customer support in either of the following ways:

- Visit the Tilera Web site at <http://www.tilera.com>.
- E-mail questions to support@tilera.com.

Notation Conventions

The following table lists notation conventions used in this guide.

Example	Description
{ }	Alternative required items in syntax descriptions appear within curly brackets. If the contents are separated by a vertical bar, you must choose one of the items.
[]	Optional items in syntax descriptions appear within brackets. If the contents are separated by a vertical bar, you must choose one of the items.
. . .	An ellipse in syntax specifies that the preceding element can be repeated an arbitrary number of times.
\$	This is the system prompt for host-side commands.
#	This is the system prompt for all Tile-side commands and for those x86 host-side commands that must be run as <code>root</code> .
<code>command</code>	A command name appears in the Courier New font
<code>computer output</code>	Computer output appears in the Courier New font.
<i>filename</i>	Non-keyword placeholders appear in italics.
GUI item	A graphical user interface (GUI) item such as a button or menu name appears in red text with the Arial font.
<i>new term</i>	An important word or phrase appears in italics.
Tile Processor™	A processor in the TILE-Gx processor™ family.
TILERA_ROOT	TILERA_ROOT is an environment variable that points to the root of the MDE installation directory. As such it is a convenient shorthand in the documentation. It is expected by many Tiler tools and examples. We recommend that you use the <code>tile-env</code> script described in <i>Getting Started with the Multicore Development Environment</i> (UG504) to set it correctly.

CHAPTER 1 UNDERSTANDING TRIO

This chapter provides an introduction to TRIO:

- [Section 1.1 Overview of TRIO](#)
- [Section 1.2 gxpci API](#)
- [Section 1.3 gxio_trio API](#)
- [Section 1.4 TRIO Context Object and GXPCI Context Object](#)
- [Section 1.5 gxpci Data Structures and Typedefs](#)
- [Section 1.6 Linking with the gxpxi and gxio Libraries](#)
- [Section 1.7 Documentation for TRIO](#)

1.1 Overview of TRIO

TRIO (transactional I/O) handles the direct memory access (DMA) data movement and buffer management between the Tile memory system and the transaction-oriented (that is, PCIe and StreamIO) subsystems.

The Tile Processor includes one or more TRIO shims, each of which contains multiple PCIe/StreamIO ports. Those ports allow PCIe and StreamIO to share the access to the MAC devices, as shown in [Figure 1](#).

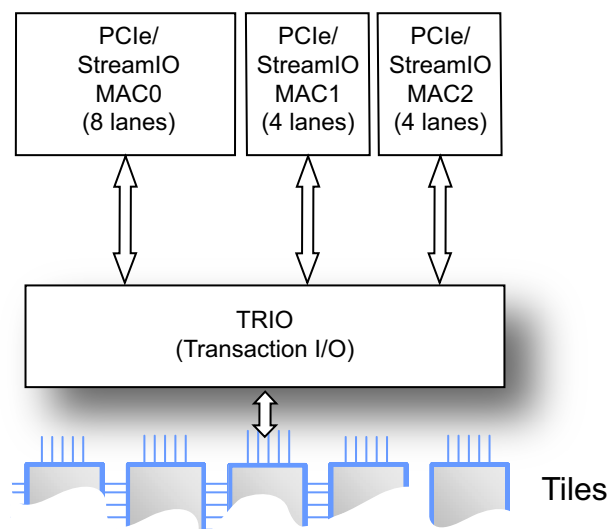


Figure 1. TRIO PCIe/StreamIO Sharing of Media Access Controllers for TILE-Gx36 Processors

Each on-chip PCIe/StreamIO port is capable of running in either of two modes:

- Endpoint mode allows the Tile Processor card to work as an add-in card that allows communication between the Linux host machine and the TILEncore-Gx card. Host machine processes communicate with Tile Linux processes through several communication mechanisms. The most prominent are:
 - Character streams: legacy syscall-based API for host communication.
 - Zero-copy command queues: legacy syscall-based API for zero-copy host communication. The zero-copy command queues enable efficient data movement because the data is copied directly from/to user-space buffers instead of being first placed into kernel/user buffers and then copied to the other side.
 - GXPCI Library: the `gxpci` API supports the zero-copy command queues and also eliminates the need for system calls on a per-transfer basis. Queue types are based on data transfer direction:
 - Host-to-Tile queue of the host Packet Queue interface (`GXPCI_PQ_H2T`)
 - Tile-to-Host queue of the host Packet Queue interface (`GXPCI_PQ_T2H`)
 - Host-to-Tile queue of the host PCIe NIC interface (`GXPCI_NIC_H2T`)
 - Tile-to-Host queue of the host PCIe NIC interface (`GXPCI_NIC_T2H`)
 - Tile-to-Tile send queue (`GXPCI_C2C_SEND`)
 - Tile-to-Tile receive queue (`GXPCI_C2C_RECV`)
- Root complex mode allows the Tile Processor to act as a host, so that system developers can connect other PCIe devices (for example, a hard disk controller) directly to the Tile Processor.

Note: Root complex applies only to PCIe. When the port operates in StreamIO mode, it is always the host.

Use StreamIO to transfer data from a TILE-Gx chip to an FPGA. You must always set the TILE-Gx chip to be the *root* or *host*.

1.2 gxpci API

The `gxpci` API is declared in `<gxpci.h>` and is implemented on top of the `gxio_trio` API. The library provides application control of data transfer operations between the various PCIe ports using the PCIe and StreamIO protocols.

The API is able to achieve low latency and high throughput by moving data directly between the user space buffers and the PCIe ports without extra memory copies, without using Linux system calls on a per-transfer basis.

Each queue is unidirectional, which allows for fine-grained hardware resource allocation.

1.3 gxio_trio API

The `gxio_trio` API, declared in `<gxio/trio.h>`, is a low-level API that allows applications to allocate and configure TRIO I/O resources such as DMA command rings, memory map windows, and device interrupts.

The `gxpci` API is easier to use than the `gxio_trio` API, which requires working knowledge of the TRIO hardware. For more information, see [Chapter 3: Using the Low-Level `gxio\_trio` API](#).

1.4 TRIO Context Object and GXPCI Context Object

All shims with a `gxio` interface have a *context* data structure to allow you to use the shim.

Each application uses a TRIO context object to help it interact with the TRIO (that is, the PCIe/StreamIO controller) hardware. The GXPCI context is used to manage application-specific resources.

If you plan to use one of the `gxpci` APIs, you must initialize a TRIO context and a GXPCI context. Use the `gxio_trio_init()` routine to initialize the TRIO context object, and the `gxpci_init()` routine to initialize the GXPCI context.

1.5 gxpci Data Structures and Typedefs

These are the data structures and typedefs that apply to applications using the `gxpci` API:

gxio_trio data structure

- `gxio_trio_context_t` (a context object used to manage TRIO hardware resources)

gxpci data structures

- `gxpci_context_t` (a structure that describes a unidirectional zero-copy queue)
- `gxpci_cmd_t` (a structure used to post zero-copy commands)
- `gxpci_comp_t` (the structure returned when a command completes)
- `pcie_c2c_send_port` (structure of registers used to specify the remote receive port to which a send port should transmit data)
- `pcie_c2c_recv_port` (structure of registers used to specify the remote send port from which a receive port receives data)
- `gxpci_resource_t` (a structure that describes the resources that are allocated to a zero-copy client)

gxpci typedefs

- `gxpci_queue_type_t` (specifies different types of zero-copy queues)

1.6 Linking with the gxpxi and gxio Libraries

All applications written to take advantage of TRIO features need to link with the `gxpci` and `gxio` libraries. Use the `-lgxpci` and `-lgxio` compiler flags.

1.7 Documentation for TRIO

Aside from this programmer's guide, see also this documentation:

- *Tile Processor I/O Device Guide for the TILE-Gx Family of Processors* (UG404)
- *Application Libraries Reference Manual* (UG527). This section:
 - “gxio Low-Level IO Device Control API”:
 - “TRIO Device API (`gxio/trio.h`)”
 - “Example Documentation”:
 - “`trio/push_dma/app.c`” – Demonstrating the use of the `gxio_trio_*` API to demonstrate push DMA between peered simulators.

- “trio/gxpci_c2c/c2c_send.c”– Demonstrating the use of the `gxpci` chip-to-chip API to send data from one PCIe port to another using peered simulators.
- “trio/gxpci_host_nic/gxpci_echo_drv_module.c”– Demonstrating how to transfer data between a tile application and a x86 host kernel module using the `gxpci` library to create sender and receiver threads.
- “trio/gxpci_host_pq/host.c”– Demonstrating how to transfer data between a tile application and a x86 host application using the TILE-Gx PCIe Packet Queue API.
- “trio/gxpci_mmi/mmi.c”– Demonstrating how to establish a memory map region's interrupts (MMI) in user-space along with `tmc/interrupt lib`.

For more information about the example applications, see [Chapter 4: Example Applications](#).

CHAPTER 2 PROGRAMMING TRIO APPLICATIONS

This chapter describes:

- [Section 2.1 TRIO Application Tasks and Routines](#)
- [Section 2.2 Transmitting Data using GXPCI Packet Queue API](#)
- [Section 2.3 Transmitting Data using the GXPCI Raw DMA API](#)

2.1 TRIO Application Tasks and Routines

When you use the `gxpci` library, your application needs to call only one `gxio` routine explicitly: `gxio_trio_init()`.

Each application needs to do these things:

1. Bind each process that will communicate from a TILE-Gx processor to a unique tile, so that each process cannot migrate to another tile during the course of its execution.
2. Call `gxio_trio_init()` to allocate a TRIO context or *service domain*.
3. Call `gxpci_init()` to allocate a GXPCI context, which includes a reference to the TRIO context. The `gxpci` context is also associated with the local PCIe port whose MAC index is passed to `gxpci_init()`. Note that one GXPCI context is associated with one process or thread that should monopolize a single tile. With multiple worker threads, they could feed into a service thread that owns the GXPCI context.
4. Call `gxpci_open_queue()` to create a data transfer queue, passing in the queue type, queue index within that type, and the remote PCIe port link index.
5. Register memory to store packet data by means of the `gxpci_iomem_register()` function.
6. Post commands using `gxpci_post_cmds()` and read back completions for those commands using `gxpci_get_comps()`.

2.2 Transmitting Data using GXPCI Packet Queue API

The Packet Queue API is intended for high performance (i.e. high throughput and low latency) transfers between the TILE-Gx (Tile) and x86 (Host) sides. To that end, the Packet Queue API avoids system calls and interrupts by, among other things, allowing user-space applications to access kernel buffers directly. Part of the Packet Queue API is implemented in the PCIe host driver.

A Packet Queue host buffer is 4MB in size because that is the maximum contiguous buffer size that is guaranteed by the kernel. The 4MB buffer is divided up into 1024 buffers of 4KB each.

A Tile-side user-space application does not need to know the address of the host buffer. The host's address is maintained in the user-space API implementation. The packet queue structure maps a portion of its state (Consumer and Producer) to the host's user-space for direct MMIO access while others are mapped to the host's kernel-space (Host buffer base address).

For more information about using the Packet Queue API, see *Managing TILE-Gx Host PCI Express Packet Queues* (AN044).

2.3 Transmitting Data using the GXPCI Raw DMA API

The raw DMA API is intended to provide the lowest latency transfers between Tile and Host. While the Packet Queue API is a sequential transfer (queue) API, the raw DMA API is a random access API.

Despite the fact that the raw DMA API is a random access API, you must allocate a kernel space buffer from the host side and use the `mmap()` routine to associate it to user space. Without doing so, a host user-space application may not get physically-contiguous buffers from the kernel, necessitating the transfer of the host buffer's address to the Tile side for every transfer and leading to increased latency and lower throughput.

Similar to the Packet Queue API, A Tile-side user-space application utilizing the raw DMA API does not need to know the address of the target (host) buffer. The host's address is maintained in the user-space API implementation. A user-space application utilizing the raw DMA API needs to simply specify the offset within the target buffer in the host. The user-space raw DMA API implementation derives the corresponding target PCI address.

Whereas the Packet Queue API uses fixed buffer sizes of 4KB each, the raw DMA API allows transfers requests ranging from 1B to 16KB.

Each Raw DMA queue allocates a single 4MB buffer on the host. It is possible to have multiple buffers per queue. The TILE-Gx application would then need to specify a buffer index/offset pair for the target buffer.

CHAPTER 3 USING THE LOW-LEVEL GXIO_TRIO API

This chapter provides an introduction to TRIO:

- [Section 3.1 Overview of gxio_trio](#)
- [Section 3.2 TRIO Ingress Hardware Resources](#)
- [Section 3.3 TRIO Egress Hardware Resources](#)
- [Section 3.4 gxio_trio Data Structures and Typedefs](#)
- [Section 3.5 Documentation for gxio_trio](#)

Note: The `gxpci` API described in [Chapter 1: Understanding TRIO](#) enables high-performance while abstracting away the low-level TRIO details. Therefore, most programmers may find using the `gxpci` API to be a better choice than using the `gxio_trio` API. In some cases, however, you might want low-level access to TRIO features. This chapter describes the `gxio_trio` API for the benefit of programmers who are interested in exploiting the low-level features of TRIO.

3.1 Overview of gxio_trio

The `gxio_trio` API, declared in `<gxio/trio.h>`, allows applications to allocate and configure TRIO I/O resources like DMA command rings, memory map windows, and device interrupts.

From the perspective of `gxio_trio`, TRIO has two major components:

- Ingress hardware resources
- Egress hardware resources

3.2 TRIO Ingress Hardware Resources

The TRIO ingress hardware is responsible for examining incoming PCIe or StreamIO packets and choosing a processing mechanism based on the packets' bus address. A programmer can use the `gxio_trio` API to configure different handlers for different ranges of bus address space. A programmer can configure "mapped memory" and "scatter queue" regions to match incoming packets within 4 KB-aligned ranges of bus addresses. Each range specifies a different set of mapping parameters to be applied when handling the ingress packet.

3.2.1 TRIO Mapped Memory (MapMem)

TRIO mapped memory (MapMem) regions allow the programmer to map incoming read and write requests directly to the application's memory space. Use the `gxio_trio_alloc_memory_maps()` routine to allocate memory mapped regions.

Given an integer MapMem number, applications can call the `gxio_trio_init_memory_map()` routine to specify the range of bus addresses that will match the region and the range of virtual addresses to which those packets will be applied.

3.2.2 TRIO Scatter Queues

TRIO scatter queue regions allow programmers to dynamically map buffers from a large address space into a small range of bus addresses. This is particularly helpful for PCIe endpoint devices, where the host generally limits the size of Base Address Registers (BARs) to tens of megabytes.

3.3 TRIO Egress Hardware Resources

The TRIO shim supports two mechanisms for egress packet generation:

- **Programmed I/O (PIO)**
PIO allows applications to create MMIO mappings for PCIe or StreamIO address space, so that the application can generate word-sized read or write transactions by issuing load or store instructions.
- **Push/pull DMA**
Push and pull DMA are tuned for larger transactions; they use specialized hardware engines to transfer large blocks of data at line rate.

3.3.1 TRIO Programmed I/O (PIO)

Programmed I/O (PIO) allows applications to create MMIO mappings for PCIe or StreamIO address space. The hardware PIO regions support access to PCIe configuration, I/O, and memory space. However, the `gxio_trio` API only supports memory space accesses.

3.3.2 TRIO Push and Pull DMA

The TRIO push and pull DMA engines allow users to copy blocks of data between application memory and the bus. Push DMA generates write packets that copy from application memory to the bus. Pull DMA generates read packets that copy from the bus into application memory.

The DMA engines are managed by means of an API that is very similar to the mPIPE eDMA interface.

3.4 `gxio_trio` Data Structures and Typedefs

These are the data structures and typedefs that apply to applications using the `gxpci` API:

`gxio_trio` data structures

- `gxio_trio_context_t` (a context object used to manage TRIO hardware resources)
- `gxio_trio_dma_queue_t` (a convenient, thread-safe interface to an egress DMA ring)

`gxio_trio` typedefs

- `gxio_trio_order_mode_t` (ordering modes for map memory regions and scatter queue regions; pointer to `gxio_trio_order_mode_e`)
- `gxio_trio_dma_desc_t` (command descriptor for push or pull DMA; pointer to `TRIO_DMA_DESC_t`)

3.5 Documentation for `gxio_trio`

See these sections of the *Application Libraries Reference Manual* (UG527):

- “`gxio` Low-Level IO Device Control API”:
 - “TRIO Device API (`gxio/trio.h`)”

- “Example Documentation”:
 - “trio/push_dma/app.c”

CHAPTER 4 EXAMPLE APPLICATIONS

This chapter describes the TRIO examples in the Examples directory at `$TILERA_ROOT/examples/trio/`:

- [Section 4.1 Chip-to-Chip Zero Copy Queue Example](#)
- [Section 4.2 Character Stream Example](#)
- [Section 4.3 Chip-to-Chip Transfer Queue Example](#)
- [Section 4.4 Host NIC Example](#)
- [Section 4.5 Host Packet Queue Example](#)
- [Section 4.6 Raw DMA Example](#)
- [Section 4.7 Memory Map Region Interrupts \(MMI\) Example](#)
- [Section 4.8 Zero Copy Example](#)
- [Section 4.9 Push DMA Example](#)
- [Section 4.10 Packet Queue with Linux Tap Example](#)

4.1 Chip-to-Chip Zero Copy Queue Example

This example shows chip-to-chip zero-copy messaging for copying data between two Tiles.

It shows how to establish a chip-to-chip data transfer queue for PCIe data transmission using two simulator instances by means of `tile-monitor`. Using the *peered simulator* commands, you configure the TRIO shim for each simulator to connect to the other simulator's TRIO shim.

The example is located here:

- `$TILERA_ROOT/examples/trio/gxpci_c2c/c2c_send.c`

This application performs the following tasks using routines from the `trio/gxpci` API:

1. Bind to a single CPU, using the `tmc_cpus_set_my_cpu()` routine.
2. Get a gxio context, by calling the `gxio_trio_init()` routine.
3. Set up a chip-to-chip send queue, transferring data from SIM0_MAC to SIM1_MAC, by calling the `gxpci_open_queue()` routine to set one to be `GXPCI_C2C_SEND` (the sender TILE PCIe port) and the other to be `GXPCI_C2C_RECV` (the receiver TILE PCIe port).
4. Allocate and register huge data buffers, using the `tmc_alloc_set_huge()`, `tmc_alloc_map()` and `tmc_alloc_get_huge_pagesize()` routines.
5. Register every allocated page into the IOTLB for TRIO DMA use, through the `gxpci_iomem_register()` routine.
6. Post a data transfer request using the `gxpci_post_cmd()` routine, after building the command of type `gxpci_cmd_t`.

7. Poll for the completion of the data transfer request with the `gxpci_get_comps()` routine.
8. Before exiting the application, release all HW and SW resources using the `gxpci_destroy()` routine.

4.2 Character Stream Example

This example demonstrates how to transfer data from host to Tile using the character stream interface.

The example is located here:

- `$TILERA_ROOT/examples/trio/gxpci_char_stream/bandwidth.c`

The example does the following:

1. Defines the `tilegxpci` device name for both the host and Tile side, the character stream index, and handles command line input to set the channel index and buffer size.
2. Opens a channel with the `open_channel()` routine.
3. Performs read and write operations using the `do_send()` and `do_recv()` routines.
4. Tests the transmission of data from the host side to the Tile processor using the `test_host_to_tile()` routine.
5. Tests the transmission of data from the host side to the Tile processor using the `test_tile_to_host()` routine.

4.3 Chip-to-Chip Transfer Queue Example

This example demonstrates how to establish a pair of data transfer queues to move data packets between two applications running on two TILE-Gx nodes. The `c2c` program creates a sender thread and a receiver thread, which communicate respectively with the receiver thread and the sender thread which runs on another TILE-Gx node using another copy of the same `c2c` program.

The example is located here:

- `$TILERA_ROOT/examples/trio/gxpci_chip_to_chip/c2c.c`

The example does the following:

1. Defines the TRIO index, remote link index, the local MAC index, and the queue indexes of the chip-to-chip send queue and receive queues.
2. Use the `gxio_trio_init()` routine to initialize the TRIO context.
3. Call the `gxio_trio_alloc_asids()` routine to get a TRIO asid, which is shared by the send and receive threads.
4. Use the `gxpci_init()` routine to initialize send and receive endpoints referencing the TRIO context, a `send_context` or `recv_context`, the TRIO index, and the local MAC index.
5. Call the `gxpci_open_duplex_queue()` routine to open a duplex queue between the local PCIe port and the remote PCIe port.
6. Use Linux `pthread_*` routines to create send and receive threads.
7. When the transfer is complete, uses the `gxpci_destroy_duplex()` routine to release the communication endpoint and free the allocated resources.

4.4 Host NIC Example

This example shows how to transfer data between a tile application and a x86 host kernel module. Using the `gxpci` library, the following application creates two threads, i.e. a sender and a receiver. The sender thread transmits packets to the host which returns the packets back to the tile, to be consumed by the receiver thread.

The example is located in this directory:

- `$TILERA_ROOT/examples/trio/gxpci_host_nic/`

The `bench.c` file the packet pool memory size, the TRIO index, the local MAC index, and a Sync variable. It parses command line arguments to fill in the MAC and bus address variables, and scan options. It manages buffers, runs the benchmark tests and reports the results.

The `gxpci_echo_drv_module.c` file is a test kernel module which echoes packets on the virtual network interface cards created by the host PCIe driver.

4.5 Host Packet Queue Example

This example shows how to transfer data between a tile application and a x86 host application using the TILE-Gx PCIe Packet Queue API.

The example is located in this directory:

- `$TILERA_ROOT/examples/trio/gxpci_host_pq/`

The `host.c` file reads a PCIe register to find how much of the ring buffer has been consumed, to determine how many data buffers are available for read for tile-to-host and how many data buffers are empty for host-to-tile. Reverse flow control is implemented by the host program writing another PCIe register to report the available space in the ring buffer.

There are also `t2h.c` and `ht2.c` files, which, respectively, transmit packets to the host and receive packets from the Tile processor.

The file `h2t.c` and `t2h.c` perform the very similar activities:

1. Bind to a single CPU using the `tmc_cpus_get_my_affinity()` routine.
2. Bind to the correct `cpu_rank` Tile in the CPU set by calling the `tmc_cpus_set_my_cpu()` and `tmc_cpus_find_nth_cpu()` routines.
3. Get a GXIO context through the `gxio_trio_init()` and `gxpci_init()` routines.
4. Open the GXIO queues in `GXPCI_RAW_DMA_RECV` mode with the `gxpci_open_queue()` routine.
5. Allocate and register data buffers through `tmc_alloc_set_huge()` and `tmc_alloc_map()` routines.
6. Register data buffers through the `gxpci_iomem_register()` routine.
7. Run the test with a `do_recv()` call on the Host side or a `do_send()` call on the Tile side.

4.6 Raw DMA Example

This example shows how to use the raw DMA API, which is intended to provide the lowest latency transfers between Tile and Host in a random access method.

The example files are located in this directory:

- `$TILERA_ROOT/examples/trio/gxpci_host_raw_dma/`

The file `h2t.c` and `t2h.c` perform the very similar activities:

1. Bind to a single CPU using the `tmc_cpus_get_my_affinity()` routine.
2. Bind to the correct `cpu_rank` Tile in the CPU set by calling the `tmc_cpus_set_my_cpu()` and `tmc_cpus_find_nth_cpu()` routines.
3. Get a GXIO context through the `gxio_trio_init()` and `gxpci_init()` routines.
4. Open the GXIO queues in `GXPCI_RAW_DMA_RECV` mode with the `gxpci_open_queue()` routine.
5. Allocate and register data buffers through `tmc_alloc_set_huge()` and `tmc_alloc_map()` routines.
6. Register data buffers through the `gxpci_iomem_register()` routine.
7. Run the test with a `do_recv()` call on the Host side or a `do_send()` call on the Tile side.

4.7 Memory Map Region Interrupts (MMI) Example

This example shows how to establish memory map region interrupts (MMI) in user space along with `tmc/interrupt` lib. Both Tile-side MMIO access and PCIe packets that fall into the `MAP_MEM_INTx` region to trigger different `MAP_MEM` interrupts are implemented, which is selected by means of a macro named `PCIE_PKT_TRIGGER`.

The example is located here:

- `$TILERA_ROOT/examples/trio/gxpci_mmi/mmi.c`

This application performs the following tasks using routines from the `trio/gxpci_mmi` API:

1. Get a `gxio` context using the `gxpci_init()` routine.
2. Allocate an Application Space Identifier (ASID) using the `gxio_trio_alloc_asids()` routine.
3. Allocate and bind a page to a memory map region using the `tmc_alloc_map()` and `gxio_trio_register_page()` routines.
4. Allocate a memory map region via the `gxio_trio_alloc_memory_maps()` routine.
5. Initialize the memory map region using the `gxio_trio_init_memory_map()` routine.
6. Allocate a PIO region through the `gxio_trio_alloc_pio_regions()` routine.
7. Target to different PCIe addresses using the `gxio_trio_init_pio_region()` routine.
8. Map the MMIO memory to PIO a region using the `gxio_trio_map_pio_region()` routine.
9. Enable the memory map interrupt for your particular memory map region using the `gxio_trio_enable_mmi()` routine.
10. Wait for the installation of the target tile IPI interrupt, by calling the `tmc_spin_barrier_wait()` routine.
11. Trigger memory map interrupts using the `gxio_trio_write_mmi_bits()` and `gxio_trio_write_uint64()` routines.

4.8 Zero Copy Example

This example demonstrates how to transfer data from host to Tile using the zero-copy queue.

The example is located here:

- `$TILERA_ROOT/examples/trio/gxpci_zero_copy/simple.c`

The example does the following:

1. Defines the zero-copy queue and parses command line arguments in order to fill in the queue index and packet size variables.
2. Opens the tile side of a host-to-tile zero-copy channel (device `/dev/trio0-mac0/h2t/N`).
3. Ensures that on the tile side, packet buffers does not cross a page boundary.
4. Registers the buffer to TRIO.
5. Constructs a receive command on the Tile side and posts it to the PCI subsystem.
6. Initiates the write command and reads back the completion, and prints the received data.
7. Opens the host side of a host-to-tile zero-copy channel (device `/dev/tilegxpci0/h2t/N`).
8. Creates a host side packet buffer from the per-channel *fast memory* region.
9. Constructs a send command from the host side of a host-to-tile channel and posts it to the PCI subsystem.
10. Initiates the write command and reads back the completion, and prints the sent data.

4.9 Push DMA Example

This example shows how to connect two simulator instances by means of `tile-monitor`. With the use of the *peered simulator* commands, each simulator's TRIO shim is configured to connect to the other simulator's TRIO shim.

This example shows TRIO's MapMem, PIO, and push DMA capabilities. Each peer runs the same code, performing several DMA operations, checking results, and using PIO to perform barriers between peers. Run two copies of the program, one on each peered simulator. Use a different MAC and bus address for each simulator.

The example is located here:

- `$TILERA_ROOT/examples/trio/push_dma/app.c`

This example includes this header file:

- `gxio/trio.h`

The example does the following:

1. Create a MapMem region that maps incoming reads and writes to the specified backing memory, using the `setup_bus_mappings()` routine.
2. Allocate a memory map region through the `gxio_trio_alloc_memory_maps()` routine.
3. Initialize that memory map region using the `gxio_trio_init_memory_map()` routine.
4. Allocate a PIO region using the `gxio_trio_alloc_pio_regions()` routine.
5. Initialize a PIO region through the `gxio_trio_init_pio_region()` routine.
6. Initialize a DMA queue to drive the push DMA ring, using the `setup_push_dma()` routine.

7. Allocate a push DMA ring by calling the `gxio_trio_alloc_push_dma_ring()` routine.
8. Bind the DMA ring to our MAC, and use registered memory to store the command ring, through the `gxio_trio_init_push_dma_queue()` routine.
9. Issue a DMA command for every possible size between 1 and 4096 bytes, using the `gxio_trio_dma_queue_put_at()` routine.
10. Wait for all the direct memory access operations to complete, by calling the `gxio_trio_dma_queue_is_complete()` routine.
11. Get a context, using the `gxio_trio_init()` routine.
12. Allocate an ASID using the `gxio_trio_alloc_asids()` routine.
13. Bind some memory using the `tmc_alloc_set_huge()` and `tmc_alloc_map()` routines.
14. Register the memory via the `gxio_trio_register_page()` routine.
15. Create MapMem and PIO regions through the `setup_bus_mappings()` routine.
16. Create push DMA queue using the `setup_push_dma()` routine.
17. Reserve a DMA descriptor slot with the `gxio_trio_dma_queue_reserve()` routine.
18. Submit a DMA request using the `gxio_trio_dma_queue_put_at()` routine.
19. Poll for DMA completion by calling the `gxio_trio_dma_queue_is_complete()` routine.

4.10 Packet Queue with Linux Tap Example

This example demonstrates how to use the TILE-Gx PCIe Packet Queue API in combination with the Linux TAP driver to implement a packet path between host kernel and tile kernel.

In order to run this example, you must configure the `tilegxpci` PCIe host driver with the number of number host-to-tile and tile-to-host packet queues, and Tile Linux with equivalent settings, by do the following:

- Install the PCIe host driver using the `tilepci-install` command and specifying the options `pq_h2t_queues=N` and `pq_t2h_queues=M`

Where:

- N is the number of host-to-tile queues
- M is the number of tile-to-host queues
- Boot the TILE Linux kernel using the following `tile-monitor` options:
 - `--hvx pci_host_pq_h2t_queues=T, P, N`
 - `--hvx pci_host_pq_t2h_queues=T, P, M`

Where:

- T is the TRIO instance number
- P is the PCIe port number
- N is the host-to-tile queue number
- M is the tile-to-host queue number

Note: The sum of tile-to-host and host-to-tile queues can not be larger than 16. <<Is the limit of 16 still true? Has tilegxpci been evolved to support up 32 queues for MDE 4.2 GA?>>

You can specify host ring buffer size and attributes that are different from these default settings. For details, see *Application Note: Managing TILE-Gx Host PCIe Packet Queues* (AN044).

Figure 1 illustrates the organization of the components included in this example.

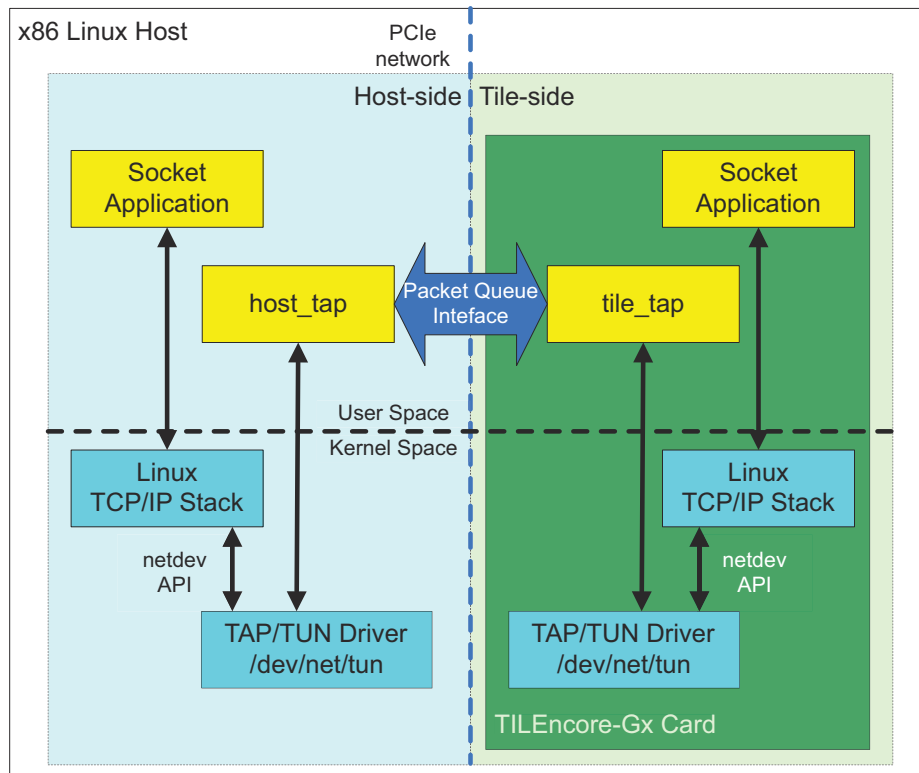


Figure 1. Components Involved in the TAP Example

The example files are located here:

- `$TILERA_ROOT/examples/trio/tap`

The Tile-side program is `tile_tap.c`, and does the following:

1. Derives the following variable settings from command line arguments:
 - PCIe MAC number (`mac_index`)
 - Processor (`cpu_rank`)
 - Interface name (`if_name`)
 - Interface address (`if_addr`)
 - Packet Queue index (`queue_index`)
 - SR-IOV Virtual Function number (`vf_instance`)
2. Bind to a single CPU using the `tmc_cpus_get_my_affinity()` routine.
3. Bind to the current Tile in the CPU set by calling the `tmc_cpus_get_my_current_cpu()` routine.
4. Get a TRIO context using the `gxio_trio_init()` routine using the `trio_context` and `trio_index`.

5. Create a communication endpoint in the host-to-tile direction using the `gxpci_init()` routine.
6. Open the PCIe queue in the host-to-tile direction using the `gxpci_open_queue()` routine.
7. Create a communication endpoint in the tile-to-host direction using the `gxpci_init()` routine.
8. Open the PCIe queue in the tile-to-host direction using the `gxpci_open_queue()` routine.
9. Use the `tmc_spin_barrier_init()` routine to initialize a barrier to a state in which no tasks have arrived at the barrier.
10. Use the Linux `pthread_create()` routine to go parallel for data transmission.
11. Call the Linux `pthread_join()` routine to wait for thread termination.
12. When you are finished with the data transfer, using the `gxpci_destroy()` routine to release the communication endpoint and free the allocated resources first in the host-to-tile and then the tile-to-host directions.

The host-side program is `host_tap.c`, and does the following:

1. Derives the following variable settings from command line arguments:
 - Interface name (`if_name`)
 - Interface address (`if_addr`)
 - Packet Queue index (`queue_index`)
 - GXPCI Card number (`card_index`)
2. Call the `tap_setup()` routine to open and configure and initialize the `/dev/net/tun` device on the host machine.
3. Use the Linux `memset()` routine to initialize memory for the host-to-tile direction.
4. Use the `pq_setup()` routine in the host-to-tile direction to use Linux function calls to open the packet queue file, map the register space, configure and allocate the DMA ring buffer, map the DMA ring, and map the status registers on the host side.
5. Use the Linux `memset()` routine to initialize memory for the tile-to-host direction.
6. Use the `pq_setup()` routine in the tile-to-host direction.
7. Use the Linux `pthread_create()` routine to go parallel for data transmission.
8. Call the Linux `pthread_join()` routine to wait for thread termination.
9. When you are finished with the data transfer, using the `pq_destroy()` routine to release the communication endpoint and free the allocated resources first in the host-to-tile and then the tile-to-host directions.

INDEX

A

audience of this guide [v](#)

B

bandwidth.c [12](#)

bench.c [13](#)

C

card_index [18](#)

changes from previous release [v](#)

compiler flags

-lgxio [3](#)

-lgxpci [3](#)

conventions

notation [vi](#)

cpu_rank [13](#)

customer support [vi](#)

D

DMA [2, 8, 15, 16](#)

document organization [v](#)

do_rcv() [12, 13, 14](#)

do_send() [12, 13, 14](#)

G

gxio/trio.h [2, 7, 15](#)

gxio_trio [2, 7, 8](#)

gxio_trio_alloc_asids [12](#)

gxio_trio_alloc_asids() [14, 16](#)

gxio_trio_alloc_memory_maps() [7, 14, 15](#)

gxio_trio_alloc_pio_regions() [14, 15](#)

gxio_trio_alloc_push_dma_ring() [16](#)

gxio_trio_dma_queue_is_complete() [16](#)

gxio_trio_dma_queue_put_at() [16](#)

gxio_trio_dma_queue_reserve() [16](#)

gxio_trio_enable_mmi() [14](#)

gxio_trio_init() [3, 5, 11, 13, 14, 16](#)

gxio_trio_init_memory_map() [7, 14, 15](#)

gxio_trio_init_pio_region() [14, 15](#)

gxio_trio_init_push_dma_queue() [16](#)

gxio_trio_map_pio_region() [14](#)

gxio_trio_register_page() [14, 16](#)

gxio_trio_write_mmi_bits() [14](#)

gxio_trio_write_uint64() [14](#)

gxpci [2](#)

gxpci.h [2](#)

GXPCI_C2C_RECV [2, 11](#)

GXPCI_C2C_SEND [2, 11](#)

gxpci_chip_to_chip [12](#)

gxpci_cmd_t() [11](#)

gxpci_destroy [18](#)

gxpci_destroy() [12](#)

gxpci_destroy_duplex [12](#)

gxpci_echo_drv_module.c [13](#)

gxpci_get_comps() [5, 12](#)

gxpci_host_raw_dma [13](#)

gxpci_init() [3, 5, 13, 14](#)

gxpci_iomem_register() [5, 11, 13, 14](#)

GXPCI_NIC_H2T [2](#)

GXPCI_NIC_T2H [2](#)

gxpci_open_duplex_queue [12](#)

gxpci_open_queue() [5, 11, 13, 14](#)

gxpci_post_cmd() [11](#)

gxpci_post_cmds() [5](#)

GXPCI_PQ_H2T [2](#)

GXPCI_PQ_T2H [2](#)

GXPCI_RAW_DMA_RECV [13, 14](#)

I

IDE online help [vi](#)

info pages [vi](#)

L

-lgxio compiler flag [3](#)

-lgxpci compiler flag [3](#)

M

mac_index [17](#)

man pages [vi](#)

MDE Document Index [vi](#)

location of [vi](#)

MDE documentation

location of [vi](#)

mmap() [6](#)

N

notation conventions [vi](#)

O

open_channel() [12](#)

organization of this guide [v](#)

P

pthread_create [18](#)
pthread_join [18](#)

R

recv_context [12](#)

S

send_context [12](#)
setup_bus_mappings() [15, 16](#)
setup_push_dma [15](#)
setup_push_dma() [16](#)
simple.c [15](#)
support
 technical or customer [vi](#)

T

tap_setup [18](#)
technical support [vi](#)
test_host_to_tile() [12](#)
test_tile_to_host() [12](#)
tmc_alloc_get_huge_pagesize() [11](#)
tmc_alloc_map() [11, 13, 14, 16](#)
tmc_alloc_set_huge() [11, 13, 14, 16](#)
tmc_cpus_find_nth_cpu() [13, 14](#)
tmc_cpus_get_my_affinity() [13, 14](#)
tmc_cpus_get_my_current_cpu [17](#)
tmc_cpus_set_my_cpu() [11, 13, 14](#)
tmc_spin_barrier_init [18](#)
tmc_spin_barrier_wait() [14](#)
trio/gxpci_mmi [14](#)
trio_context [17](#)
trio_index [17](#)